

ARCHITECTURE DESIGN DOCUMENT (ADD): CSVAT

Aaman Sheikh — M Rayhan Khan — Saksham Kushwaha

1 Introduction

1.1 Purpose

The CORE Stack Village Analytics Tool (CSVAT) is a web-based geospatial analytics platform. It allows practitioners, researchers, and planners to generate village-level socio-ecological insights (Cropping Intensity, Surface Water, Vegetation) using CORE Stack datasets.

Unlike traditional GIS desktop tools, CSVAT offers a zero-setup browser interface with two execution modes: Server-Side (Cloud) for heavy batch processing and Client-Side (WASM) for rapid analysis with reduced server dependency.

1.2 Scope

- **Web Interface:** React-based UI for boundary selection and report visualization.
- **Dual Execution:** Support for analytics running on a Python Backend (Celery/GEE) or directly in the browser (WebAssembly).
- **Reporting:** Generation of interactive Data Stories (HTML) and static exports (PDF/CSV).
- **Data Integration:** Resolution of administrative boundaries via CORE Stack APIs/GEE and fetching raster data from CORE Stack.

2 System Architecture

2.1 Architectural Pattern

CSVAT follows a Client-Server Architecture with a Microservices-lite approach for the backend. The system is designed to be "execution-agnostic," meaning the logic for analytics is decoupled from the environment where it runs.

2.2 Architecture & Workflow Diagrams

- System Architecture
- Execution Workflow (Sequence): Attached below

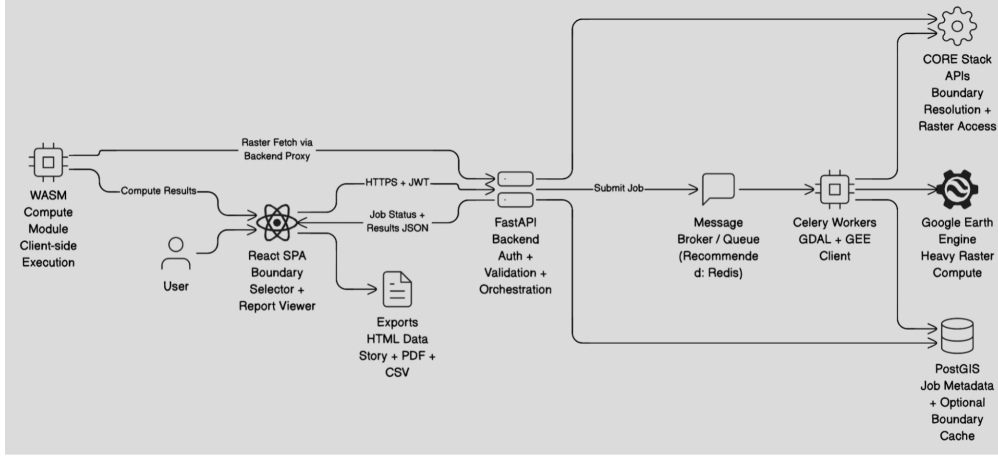


Figure 1: System Architecture Diagram (flowchart LR)

2.3 Component Description

- **Presentation Layer (Client):** A React.js Single Page Application (SPA). It handles user interactions, renders maps using a standard web mapping library (TBD), and visualizes charts using a React charting library (TBD). It includes a WASM Compute Module for client-side processing.
- **API Gateway / Application Layer:** A FastAPI (Python) service that handles authentication, request validation, and orchestrates job dispatch.
- **Async Worker Layer:** Celery workers (Recommended: backed by Redis). These workers interface with Google Earth Engine (GEE) and GDAL to process server-side jobs.
- **Data Persistence Layer:** PostgreSQL + PostGIS for storing job metadata and optionally caching resolved administrative boundaries.
- **External Services:**
 - CORE Stack APIs: Source of truth for administrative boundaries, raster URLs, and ancillary data.
 - Google Earth Engine (GEE): Computational engine for heavy raster arithmetic (Server-Mode).

3 Execution Modes & Data Flow (Architectural View)

The system supports two distinct workflows. Both execution modes produce equivalent output structures (JSON → HTML/PDF/CSV), ensuring consistent results across environments.

3.1 Mode A: Server-Side Execution

Used for high-precision, heavy compute, or batch jobs.

1. **Submit:** Client sends POST /api/v1/jobs with {village_id, analytics_type}.

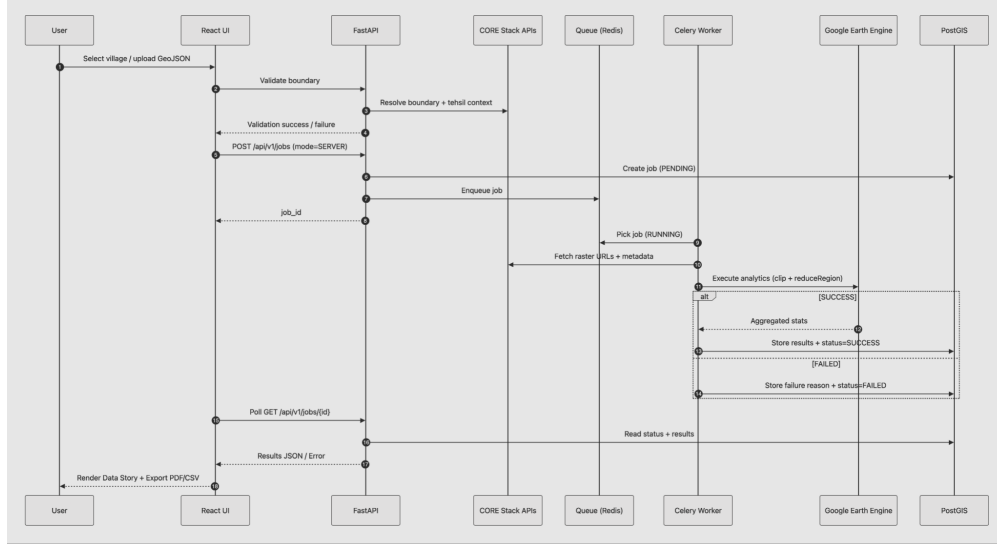


Figure 2: Server-Side Execution Sequence Diagram

2. **Queue:** FastAPI pushes task to the message broker/queue (Recommended: Redis); returns `job_id` (Status: PENDING).
3. **Process:**
 - Celery Worker picks up task (Status: RUNNING).
 - Worker resolves Village Polygon via CORE Stack/GEE (optionally checking PostGIS cache).
 - Worker acts as GEE Client → Sends computation request to Google Earth Engine.
 - GEE returns statistical aggregations (JSON) or clipped Raster URLs.
4. **Store:** Worker saves results to the persistence layer and updates Status to SUCCESS.
5. **Poll & Render:** Client polls GET `/api/v1/jobs/{id}`, receives JSON, and renders the "Data Story."

3.2 Mode B: Client-Side Execution (WASM)

Used for rapid prototyping, lower latency, or reducing server costs.

- **Boundary Resolution:** Client initiates boundary resolution (authoritative lookup via Backend APIs to protect tokens) or uploads GeoJSON.
- **Data Fetch:** Client requests signed URLs for Rasters via Backend Proxy (avoids exposing CORE Stack auth tokens in browser).
- **Compute:**
 - Browser loads the Client-side compute module implemented in WebAssembly.
 - Rasters are fetched and array buffers are processed in-memory within the browser.
- **Render:** Results are passed directly to the React Visualization component.
- **Sync (Optional):** User can choose to "Save" the run, sending metadata to the Backend.

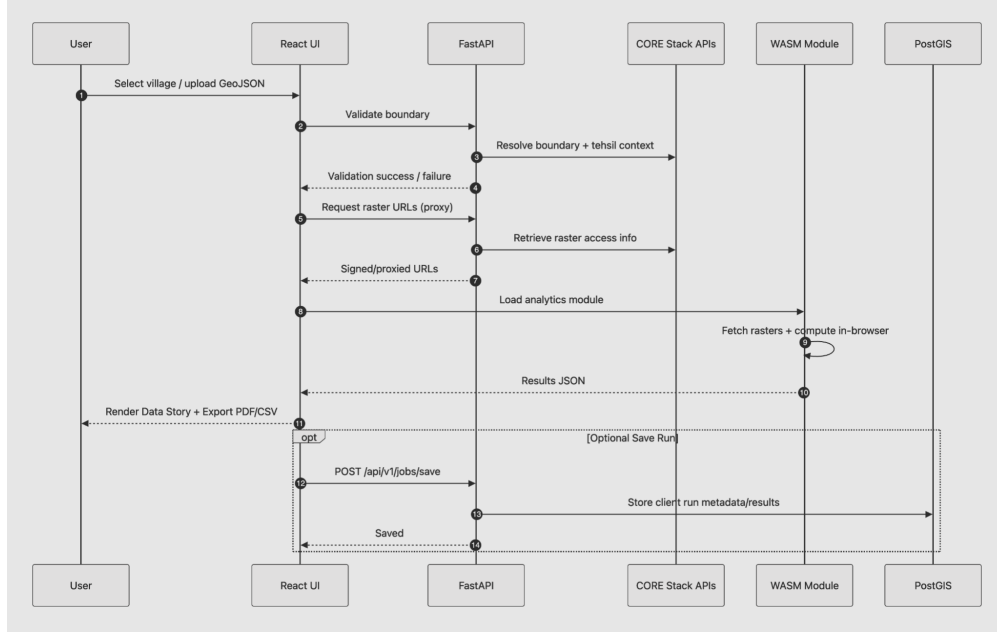


Figure 3: Client-Side WASM Execution Sequence Diagram

4 Connectors and Communication Mechanisms

CSVAT components communicate using well-defined connectors. Each connector enforces separation of concerns and ensures secure and reliable interaction between system layers.

4.1 HTTPS (REST Connector)

Type: Synchronous Request–Response

Used Between:

- React Frontend ↔ FastAPI Backend
- FastAPI Backend ↔ CORE Stack APIs

Purpose:

- Boundary resolution
- Job submission
- Job status polling
- Retrieval of raster metadata
- Delivery of analytics results

All communication occurs over HTTPS. Authentication is enforced using JWT tokens. Backend services act as a proxy for sensitive CORE Stack API calls to prevent exposure of credentials in the client.

4.2 Message Queue Connector (Redis Broker)

Type: Asynchronous Messaging

Used Between:

- FastAPI API Layer ↔ Celery Worker Layer

Purpose:

- Dispatch heavy raster-processing tasks
- Decouple HTTP request lifecycle from compute lifecycle
- Prevent request timeouts

When a job is submitted, FastAPI publishes a task to Redis. Celery workers consume tasks independently and update job status in PostGIS.

4.3 Database Connector (PostgreSQL / PostGIS)

Type: Persistent Data Connector

Used Between:

- FastAPI ↔ PostGIS
- Celery Workers ↔ PostGIS

Purpose:

- Store job metadata
- Store job status (PENDING, RUNNING, SUCCESS, FAILED)
- Store final analytics JSON
- Optional caching of resolved boundaries

Database communication uses secure internal network connections within the deployment environment.

4.4 WebAssembly Execution Connector

Type: In-Browser Compute Connector

Used Between:

- React UI ↔ WASM Compute Module

Purpose:

- Execute raster clipping and aggregation in-browser
- Process raster ArrayBuffers locally
- Return computed JSON results to UI

This connector avoids server round-trips for analytics in client-side mode.

4.5 Proxy Connector for Raster Access

Type: Secure Proxy Connector

Used Between:

- React Frontend → FastAPI → CORE Stack

Purpose:

- Provide signed or proxied raster URLs
- Protect CORE Stack API keys
- Enforce access control

The frontend never directly communicates with CORE Stack APIs.

4.6 Polling Connector (Job Status Monitoring)

Type: Repeated REST Polling

Used Between:

- React Frontend ↔ FastAPI Backend

Purpose:

- Monitor asynchronous job status
- Retrieve final JSON results once processing completes

Polling frequency is controlled to prevent server overload.

5 Non-Functional Requirements (Architectural Strategy)

- **Performance:** Server Mode: Asynchronous processing prevents timeouts. WASM Mode: Offloads compute to client, zero server latency.
- **Reproducibility:** Job parameters and input Raster versions are stored in the job metadata (PostGIS) to ensure historical traceability.
- **Security:** Auth: HTTPS for all communications. API keys/JWTs managed by backend. No secrets exposed in frontend code.
- **Scalability:** Backend is a stateless container (Docker). Workers scale horizontally based on queue length.

6 Deployment Architecture

- **Hosting:** AWS/GCP or On-Premise Linux Server (Proposed).
- **Containerization:** Docker Compose for MVP (FastAPI, Redis, Worker, PostGIS) - (Proposed).
- **Frontend:** Served via Nginx or Object Storage/CDN - (Proposed).
- **CI/CD:** GitHub Actions to build Docker images and deploy to staging.